

TARGET FILE	TOTAL LINES	LANGUAGE	DOMAIN
retriever.py	179 Lines	Python 3.x	RAG / Case Retrieval

Role in the project's RAG system: This file is the **Retrieval** layer. It reads the stored case files (cases/*.json), turns each case into a Term-Frequency (TF) vector and keeps them in memory. When a new suspect or case description arrives, `search_similar_cases()` vectorizes it, ranks all stored cases by cosine similarity and returns the top matches with metadata and a text snippet. That retrieved context is passed to the generation/analysis stage so answers are grounded in real, similar past cases. It needs no external libraries.

#	SOURCE CODE	TECHNICAL EXPLANATION
1	<code>"""</code>	Opens the module-level docstring (triple quotes). It documents the whole file and is the first thing shown by <code>help()</code> or IDE tooltips.
2	<code>rag/retriever.py</code>	Docstring line naming the file path <code>rag/retriever.py</code> . This file is the Retrieval stage of the project's RAG (Retrieval-Augmented Generation) pipeline.
3		Blank line inside the docstring, separating the file name from the description.
4	<code>Pure-Python TF/cosine similarity retriever – zero external dependencies.</code>	States the design goal: a pure-Python TF/cosine-similarity retriever with zero external dependencies. No vector database or embedding API is needed to fetch similar past cases.
5	<code>"""</code>	Closing triple quotes that end the module docstring.
6		Blank spacer line separating the docstring from the import section.
7	<code>import os</code>	Imports the operating-system interface module, used for building absolute paths, checking directories and listing the case files.
8	<code>import json</code>	Imports the JSON library, used to parse each stored case file (cases/*.json) into a Python dictionary.
9	<code>import math</code>	Imports the math module; <code>math.sqrt</code> is used to compute vector magnitudes for cosine similarity.
10	<code>import re</code>	Imports the regular-expression module, used by the tokenizer to split text into lowercase words and numbers.
11	<code>from typing import List, Dict, Any, Optional</code>	Imports type-hint constructs (List, Dict, Any, Optional) for clear, self-documenting function signatures.
12		Blank line (PEP 8 spacing) between the imports and the first function.
13		Second blank line; PEP 8 asks for two blank lines before top-level definitions.
14	<code>def _tokenize(text: str) -> List[str]:</code>	Defines the private helper <code>_tokenize(text)</code> , which converts raw text into a list of word tokens. This is the first step of turning text into a searchable vector.
15	<code> return re.findall(r"[a-z0-9]+", str(text).lower())</code>	Converts the input to a string, lowercases it, and extracts every run of letters/digits with <code>re.findall</code> . Punctuation and spaces are discarded, so "Theft, 2021" becomes <code>['theft', '2021']</code> .
16		Blank spacer line between functions.
17		Second blank spacer line (PEP 8).
18	<code>def _tf(tokens: List[str]) -> Dict[str, float]:</code>	Defines <code>_tf(tokens)</code> , which builds a Term-Frequency vector: a dict mapping each word to its relative frequency in the text.

#	SOURCE CODE	TECHNICAL EXPLANATION
19	<code>counts: Dict[str, int] = {}</code>	Creates an empty dictionary counts to hold the raw number of times each token appears (typed as str to int).
20	<code>for token in tokens:</code>	Loops over every token in the token list.
21	<code>counts[token] = counts.get(token, 0) + 1</code>	Increments the counter for the current token; counts.get(token, 0) returns 0 the first time a word is seen.
22	<code>total = len(tokens) or 1</code>	Stores the total token count; or 1 prevents division by zero when the list is empty.
23	<code>return {token: count / total for token, count in counts.items()}</code>	Returns a dict comprehension that divides every raw count by the total, producing normalized frequencies (0 to 1). This dict is the sparse vector used for similarity search.
24		Blank spacer line between functions.
25		Second blank spacer line (PEP 8).
26	<code>def _cosine(a: Dict[str, float], b: Dict[str, float]) -> float:</code>	Defines _cosine(a, b), which measures how similar two TF vectors are. It returns a float between 0.0 (nothing in common) and 1.0 (identical direction).
27	<code>if not a or not b:</code>	Guard clause: if either vector is empty, similarity cannot be computed.
28	<code>return 0.0</code>	Returns 0.0 (no similarity) for the empty-vector case.
29	<code>keys = set(a) & set(b)</code>	Builds the set of words that appear in both vectors (set intersection). Only shared words contribute to the dot product.
30	<code>if not keys:</code>	Checks whether the two texts share no words at all.
31	<code>return 0.0</code>	Returns 0.0 immediately because there is no overlap, which saves the magnitude computation.
32	<code>dot = sum(a[k] * b[k] for k in keys)</code>	Computes the dot product: the sum of a[k] * b[k] over all shared words.
33	<code>mag_a = math.sqrt(sum(value * value for value in a.values()))</code>	Computes the magnitude (Euclidean length) of vector a: the square root of the sum of squared weights.
34	<code>mag_b = math.sqrt(sum(value * value for value in b.values()))</code>	Computes the magnitude of vector b in the same way.
35	<code>if mag_a == 0 or mag_b == 0:</code>	Safety check: a zero magnitude would cause division by zero.
36	<code>return 0.0</code>	Returns 0.0 if either magnitude is zero.
37	<code>return dot / (mag_a * mag_b)</code>	Returns the cosine similarity formula: dot product divided by the product of both magnitudes. This score ranks stored cases against the query.
38		Blank spacer line before the class definition.
39		Second blank spacer line (PEP 8).
40	<code>class CaseRetriever:</code>	Declares the CaseRetriever class, the main retriever of the RAG system. It loads case files, indexes them in memory and answers similarity searches.
41		Blank line after the class header, before the first method.
42	<code>def __init__(self, cases_dir: Optional[str] = None, db_path: str = ".chroma_db"):</code>	Constructor. cases_dir is an optional folder of case JSON files (defaults to None). db_path defaults to .chroma_db; it is kept for compatibility with a vector-DB style setup and is only stored here.

#	SOURCE CODE	TECHNICAL EXPLANATION
43	<code>if cases_dir is None:</code>	Checks whether the caller supplied no cases folder.
44	<code>self.cases_dir = os.path.abspath(</code>	Default branch: starts computing an absolute path with <code>os.path.abspath(...)</code> for the cases folder.
45	<code>os.path.join(os.path.dirname(__file__), "..",</code> <code>"cases")</code>	Joins this file's directory (<code>os.path.dirname(__file__)</code>) with <code>..</code> and <code>cases</code> , so the default location is the project's <code>cases/</code> folder next to the <code>rag/</code> folder.
46	<code>)</code>	Closes the <code>os.path.abspath(</code> call opened on line 44.
47	<code>elif os.path.isabs(cases_dir):</code>	Else-if branch: a <code>cases_dir</code> was given; checks whether it is already an absolute path.
48	<code>self.cases_dir = cases_dir</code>	Uses the given absolute path as-is.
49	<code>else:</code>	Final else branch: the given path is relative.
50	<code>self.cases_dir = os.path.abspath(cases_dir)</code>	Converts the relative path to an absolute path so the retriever works no matter which directory the program is started from.
51	<code>self.db_path = db_path</code>	Stores <code>db_path</code> on the instance (unused for the in-memory search).
52	<code>self._docs: List[Dict[str, Any]] = []</code>	Creates the in-memory index <code>self._docs</code> , an empty list of document records. Each record will hold case id, text, TF vector and metadata.
53	<code>self._index_cases()</code>	Builds the index immediately at start-up by calling <code>_index_cases()</code> , so the retriever is ready to search after creation.
54		Blank line separating the constructor from the next method.
55	<code>def reload(self) -> int:</code>	Defines <code>reload()</code> , which returns an int (the number of indexed documents). It is called when new case files are added or changed.
56	<code>"""Rebuild the in-memory index after case files change."""</code>	Docstring: rebuilds the in-memory index after case files change (for example when new evidence or a case is uploaded).
57	<code>self._docs = []</code>	Clears the old index by assigning a fresh empty list.
58	<code>self._index_cases()</code>	Re-reads all case files and rebuilds the index.
59	<code>return len(self._docs)</code>	Returns how many cases are now indexed, so the caller can confirm the reload worked.
60		Blank line separating methods.
61	<code>@staticmethod</code>	Decorator <code>@staticmethod</code> : the next method needs no <code>self</code> , so it works purely on the case dictionary passed in.
62	<code>def _case_text(case: Dict[str, Any]) -> str:</code>	Defines <code>_case_text(case)</code> , which converts one case dictionary into one plain-text string. This text is what gets tokenized and searched.
63	<code>"""Flatten common case fields so uploaded evidence is</code> <code>searchable."""</code>	Docstring: flattens the common case fields so uploaded evidence is searchable.
64	<code>def flatten(value: Any) -> str:</code>	Defines the nested helper <code>flatten(value)</code> , which turns any value (string, list, dict, number) into text.
65	<code>if isinstance(value, list):</code>	Checks whether the value is a list (for example a list of evidence items or suspects).

#	SOURCE CODE	TECHNICAL EXPLANATION
66	<code>return ", ".join(flatten(item) for item in value)</code>	Recursively flattens each list item and joins them with commas into a single string.
67	<code>if isinstance(value, dict):</code>	Checks whether the value is a dictionary (nested structured data).
68	<code>return ", ".join(</code>	Starts joining the dictionary's entries with commas (opens the <code>", ".join(</code> call).
69	<code> f"{key}: {flatten(item)}"</code>	Formats each entry as <code>key: flattened value</code> , keeping both the label and content searchable.
70	<code> for key, item in value.items()</code>	Generator clause that iterates over all key/value pairs of the dictionary.
71	<code>)</code>	Closes the <code>join(</code> call from line 68.
72	<code>return str(value or "")</code>	Base case: converts any other value to a string; <code>value or ""</code> turns <code>None/False/empty</code> into an empty string.
73		Blank line between the nested helper and the field list.
74	<code>fields = (</code>	Starts the <code>fields</code> tuple: the case keys that will be included in the searchable text.
75	<code> "title", "summary", "crime_type", "location", "status",</code>	Fields: title, summary, crime type, location and status of the case.
76	<code> "modus_operandi", "personality_disorder",</code> <code> "common_traits",</code>	Fields: modus operandi, personality disorder and common traits, the behavioural profile of the offender.
77	<code> "evidence", "suspects", "case_notes",</code> <code> "investigation_notes",</code>	Fields: evidence, suspects, case notes and investigation notes.
78	<code>)</code>	Closes the <code>fields</code> tuple.
79	<code>return " ".join(</code>	Returns one string, joining all field strings with spaces (opens the <code>" ".join(</code> call).
80	<code> f"{field.replace('_', ' ').title()}: {flatten(case.get(field, ''))}."</code>	For each field, builds <code>Field Name: value</code> . The name is prettified (underscores become spaces, Title Case) and the value is flattened. The label words also become searchable tokens.
81	<code> for field in fields</code>	Generator loop over each field name in the <code>fields</code> tuple.
82	<code> if case.get(field)</code>	Filter: only includes a field if the case actually has a non-empty value for it.
83	<code>)</code>	Closes the <code>join(</code> call from line 79.
84		Blank line between methods.
85	<code>def _index_cases(self) -> None:</code>	Defines <code>_index_cases()</code> , the method that reads every case file and builds the searchable in-memory index. Returns <code>None</code> .
86	<code>if not os.path.isdir(self.cases_dir):</code>	Checks that the cases directory exists; <code>os.path.isdir</code> returns <code>False</code> for missing paths.
87	<code>return</code>	Exits quietly if there is no cases folder, leaving the index empty instead of crashing.
88		Blank line separating the directory check from the file loop.
89	<code>for file in os.listdir(self.cases_dir):</code>	Loops over every file name inside the cases folder.
90	<code>if not file.lower().endswith(".json"):</code>	Skips any file that does not end with <code>.json</code> (case-insensitive).

#	SOURCE CODE	TECHNICAL EXPLANATION
91	<code>continue</code>	Moves to the next file.
92		Blank line before the path construction.
93	<code>file_path = os.path.join(self.cases_dir, file)</code>	Builds the full path of the current JSON file.
94	<code>try:</code>	Starts a try block so a broken or unreadable file cannot stop the whole indexing process.
95	<code>with open(file_path, "r", encoding="utf-8") as f:</code>	Opens the file for reading with UTF-8 encoding inside a with block (file closes automatically).
96	<code>content = f.read().strip()</code>	Reads the whole file and strips leading/trailing whitespace.
97	<code>if not content:</code>	Checks whether the file is empty.
98	<code>continue</code>	Skips empty files.
99		Blank line separating reading from parsing.
100	<code>case = json.loads(content)</code>	Parses the JSON text into a Python object (case).
101	<code>if not isinstance(case, dict):</code>	Validates that the parsed JSON is a dictionary (one case record).
102	<code>continue</code>	Skips files whose JSON is a list, string or other non-dict.
103		Blank line before text preparation.
104	<code>doc_text = self._case_text(case)</code>	Calls <code>_case_text(case)</code> to flatten the case into one searchable text string, <code>doc_text</code> .
105		Blank line between text creation and tokenization.
106	<code>tokens = _tokenize(doc_text)</code>	Tokenizes the case text into lowercase word tokens.
107	<code>if not tokens:</code>	Checks whether tokenization produced nothing.
108	<code>continue</code>	Skips cases with no searchable words.
109		Blank line before the metadata extraction.
110	<code>case_id = str(case.get("case_id", file))</code>	Reads <code>case_id</code> , falling back to the file name if missing, and converts it to a string.
111	<code>title = str(case.get("title", "Unknown"))</code>	Reads the case title, defaulting to "Unknown".
112	<code>location = str(case.get("location", "Unknown"))</code>	Reads the case location, defaulting to "Unknown".
113	<code>crime_type = str(case.get("crime_type", "Unknown"))</code>	Reads the <code>crime_type</code> , defaulting to "Unknown". Lines 110 to 113 collect the metadata shown later in search results.
114		Blank line before appending to the index.
115	<code>self._docs.append({</code>	Appends a new document record (dictionary) to the in-memory index <code>self._docs</code> .
116	<code> "case_id": case_id,</code>	Record key <code>case_id</code> : unique identifier of the case.
117	<code> "text": doc_text,</code>	Record key <code>text</code> : the full flattened text, used later to create the result snippet.

#	SOURCE CODE	TECHNICAL EXPLANATION
118	<code>"tf": _tf(tokens),</code>	Record key tf: the precomputed Term-Frequency vector, so it is calculated once at index time, not on every search.
119	<code>"metadata": {</code>	Record key metadata: opens a nested dictionary with descriptive data about the case.
120	<code>"case_id": case_id,</code>	Metadata: case_id.
121	<code>"title": title,</code>	Metadata: title.
122	<code>"case_title": title,</code>	Metadata: case_title, a duplicate key so callers can read either name.
123	<code>"location": location,</code>	Metadata: location.
124	<code>"crime_type": crime_type,</code>	Metadata: crime_type.
125	<code>},</code>	Closes the metadata dictionary.
126	<code>})</code>	Closes the record dictionary and the <code>append()</code> call. The case is now indexed.
127		Blank line before the exception handler.
128	<code>except (json.JSONDecodeError, OSError, TypeError, ValueError):</code>	Catches expected failures: bad JSON, file errors (<code>OSError</code>), wrong types and bad values, so one damaged file is skipped instead of breaking the RAG system.
129	<code>continue</code>	Silently skips the bad file and continues with the next one in the loop.
130		Blank line separating the indexing method from the search method.
131	<code>def search_similar_cases(</code>	Defines <code>search_similar_cases</code> , the main public RAG retrieval method. Returns a list of result dictionaries.
132	<code>self,</code>	First parameter is <code>self</code> (the retriever instance).
133	<code>suspect_query: str,</code>	<code>suspect_query</code> is the text describing the current suspect or case (behaviour, crime type, traits) that is searched against the stored cases.
134	<code>top_k: int = 3</code>	<code>top_k</code> is how many best matches to return; it defaults to 3.
135	<code>) -> List[Dict[str, Any]]:</code>	Closes the signature. Return type is <code>List[Dict[str, Any]]</code> .
136	<code># Every call creates a fresh query vector from the CURRENT suspect.</code>	Comment: every call builds a new query vector from the current suspect, so results never carry over from a previous search.
137	<code>if not suspect_query or not str(suspect_query).strip():</code>	Checks that the query is not empty or whitespace-only.
138	<code>return []</code>	Returns an empty list for an empty query.
139	<code>if not self._docs:</code>	Checks whether the index has no documents.
140	<code>return []</code>	Returns an empty list because there is nothing to search.
141		Blank line before the <code>top_k</code> validation.
142	<code>try:</code>	Starts a <code>try</code> block to sanitize <code>top_k</code> .

#	SOURCE CODE	TECHNICAL EXPLANATION
143	<code>top_k = max(1, int(top_k))</code>	Converts <code>top_k</code> to an integer and forces it to be at least 1 with <code>max(1, ...)</code> .
144	<code>except (TypeError, ValueError):</code>	Catches invalid <code>top_k</code> values such as <code>None</code> or non-numeric text.
145	<code>top_k = 3</code>	Falls back to the default of 3 results.
146		Blank line before query processing.
147	<code>query_tokens = _tokenize(str(suspect_query))</code>	Tokenizes the query text using the same <code>_tokenize</code> function used for the case documents, so both sides are comparable.
148	<code>if not query_tokens:</code>	Checks whether the query produced no usable tokens.
149	<code>return []</code>	Returns an empty list if the query has no words or digits.
150		Blank line before vectorizing the query.
151	<code>query_tf = _tf(query_tokens)</code>	Builds the query's TF vector, the vector-space representation of the suspect query.
152	<code>scored = []</code>	Creates an empty list <code>scored</code> to collect candidate matches with their scores.
153		Blank line before the scoring loop.
154	<code>for doc in self._docs:</code>	Loops over every indexed case document.
155	<code>similarity = _cosine(query_tf, doc["tf"])</code>	Computes the cosine similarity between the query vector and the case's precomputed vector.
156	<code>if similarity <= 0:</code>	Checks for zero similarity (no shared words).
157	<code>continue</code>	Skips irrelevant cases so they never appear in the results.
158	<code>distance = max(0.0, 1.0 - similarity)</code>	Computes <code>distance = 1 - similarity</code> (clamped at 0.0), giving a distance-style metric similar to vector-database output. Lower distance means a better match.
159	<code>scored.append((similarity, distance, doc))</code>	Adds a tuple of similarity, distance and the document to the <code>scored</code> list.
160		Blank line before sorting.
161	<code># Highest similarity = best match.</code>	Comment: the highest similarity means the best match.
162	<code>scored.sort(key=lambda item: item[0], reverse=True)</code>	Sorts candidates in place by the similarity score (first tuple item), highest first.
163		Blank line before building the output.
164	<code>results: List[Dict[str, Any]] = []</code>	Creates the empty <code>results</code> list that will be returned to the caller.
165	<code>for similarity, distance, doc in scored[:top_k]:</code>	Loops over only the top <code>top_k</code> scored matches (slice <code>[:top_k]</code>), unpacking similarity, distance and document.
166	<code>metadata = dict(doc["metadata"])</code>	Makes a copy of the document's metadata so the stored index cannot be changed accidentally.
167	<code>results.append({</code>	Appends one result dictionary to <code>results</code> (opens the <code>append({</code> call).
168	<code>"case_id": doc["case_id"],</code>	Result field <code>case_id</code> : which case matched.

#	SOURCE CODE	TECHNICAL EXPLANATION
169	<code>"metadata": metadata,</code>	Result field metadata: the full metadata copy.
170	<code> "case_title": metadata.get("case_title", metadata.get("title", "Unknown Case")),</code>	Result field case_title: read from metadata, falling back to title, then to "Unknown Case".
171	<code> "location": metadata.get("location", "Unknown"),</code>	Result field location of the matched case.
172	<code> "crime_type": metadata.get("crime_type", "Unknown"),</code>	Result field crime_type of the matched case.
173	<code> "snippet": doc["text"][:300],</code>	Result field snippet: the first 300 characters of the case text, a short preview that can be passed to the LLM as context.
174	<code> "distance": distance,</code>	Result field distance: how far the case is from the query (0 is closest).
175	<code> "similarity": similarity,</code>	Result field similarity: cosine score (higher is better).
176	<code> })</code>	Closes the result dictionary and the append call.
177		Blank line before the return statement.
178	<code> return results</code>	Returns the ranked list of similar cases. In the RAG flow, this retrieved context is then given to the generation/analysis step (LLM) so its answer is grounded in real past cases.
179		End-of-file marker: the trailing empty line after the final statement (the newline that ends the file, counted as line 179 by the editor). It contains no code.

Flow summary (lines 14 to 178): `_tokenize` (text to words) → `_tf` (words to TF vector) → `CaseRetriever._index_cases` (case files to in-memory index) → `search_similar_cases` (query vector, `_cosine` ranking, top-k results for the LLM). All 179 lines of the file are covered above, including blank lines and the end-of-file line.